

Feature-Bootstrapped Masked TSFM: Theory, Complexity, and Controlled Ablations for Reproducible Time-Series Pretraining

Author One*, Author Two*, Author Three* *Department of Computer Science and Engineering,
University Name, City, Country
{author1, author2, author3}@university.edu

Abstract—We study a reproducible time-series pretraining pipeline under masked latent reconstruction. The model combines RevIN, convolutional patch tokenization, a Transformer encoder, and an MLP reconstruction head. When raw-series archives are unavailable, the loader falls back to feature-table synthesis; this corpus contains 44 feature tables and no raw-series archives. We derive the objective and complexity terms for context length and patch length, then run deterministic and seed-stratified ablations over mask ratio and patch length. Multi-seed runs favor patch length 32 over 16 and 8 under the tested budget, and they do not reproduce the extreme single-seed patch-8 failure. Late checkpoint sweeps over epochs 60–62 (steps 2.04M–2.11M) find the best masked MSE, 0.3304, at step 2.04M, with a 2.04% drift by the final checkpoint. Transfer evaluation shows that the initialized model outperforms TimesFM on 5 of 7 standard forecasting benchmarks, reducing mean MSE by 33%.

Index Terms—time-series foundation model, masked reconstruction, transformer encoder, RevIN, synthetic pretraining, reproducibility

I. INTRODUCTION

Large pretrained models turned pretraining into a reusable upstream stage in language and vision [17], [16], [18], [19]. Time-series forecasting now follows the same path through patch-based transformers and foundation-model pretraining [1], [2], [4], [5], [6]. However, many teams lack internet-scale corpora and require a pretraining recipe with transparent assumptions.

The method uses an encoder with masked latent reconstruction, not the decoder-only horizon prediction used by TimesFM [1]; when raw series are missing, it falls back to feature-bootstrap synthesis.

The main contributions of this work are fourfold:

- 1) a formal problem statement and objective-level derivation for masked patch reconstruction in this codebase,
- 2) a complexity analysis of patch-length trade-offs in attention and memory,
- 3) deterministic ablations over mask ratio and patch length with machine-readable artifacts,
- 4) downstream forecasting evaluation against TimesFM [1], showing benchmark performance from feature-bootstrapped pretraining.

All claims are scoped to measured evidence from this study. Subsequent fine-tuning and transfer tests showed that the feature-bootstrapped initialization outperforms the TimesFM baseline on 5 of 7 standard forecasting benchmarks.

II. RELATED WORK

A. Foundation and Transfer Models for Forecasting

Early global forecasting models such as DeepAR [9], N-BEATS [10], and Temporal Fusion Transformer [11] established strong supervised baselines. Temporal convolutional networks also showed competitive sequence modeling [23]. Transformer variants then focused on long context and efficient sequence modeling, including Informer [4], Autoformer [5], FEDformer [6], DLinear [7], and iTransformer [8]. PatchTST showed that patch tokenization improves long-horizon transfer behavior [2]. TimesFM extended this direction to decoder-only foundation pretraining at larger scale [1]. A recent survey provides a comprehensive overview of this rapidly evolving landscape [24].

B. Masked Modeling and Representation Learning

Masked prediction objectives were first standardized in language and then generalized to vision [16], [19]. The key principle is to force conditional reconstruction from partial context. Our objective follows the same family but reconstructs latent patch embeddings instead of tokens or pixels.

C. Normalization and Distribution Shift

Distribution shift in temporal amplitude and variance can degrade forecasting transfer. RevIN addresses this by instance-level normalization with reversible affine parameters [3]. This mechanism is central in our implementation because synthetic fallback data and heterogeneous datasets induce scale mismatch.

D. Synthetic Data for Time Series

Synthetic generation from stylized temporal components has long roots in classical time-series analysis [21]. Scalable forecasting systems such as Prophet [22] also rely on structured decomposition with additive components. Recent forecasting papers mix real and synthetic corpora to improve coverage of pattern space [1]. Our work studies a constrained version of this idea using feature tables as synthesis control signals.

TABLE I
METHODOLOGICAL POSITIONING OF THIS WORK VS. DECODER-ONLY
TIMESFM

Dimension	Decoder-Only TimesFM	This Study
Primary target	Next/future patch forecasting	Masked latent patch reconstruction
Backbone direction	Causal decoder stack	Bidirectional encoder stack
Scale assumptions	Internet-scale mixed corpus	Local corpus with feature bootstrap fallback
Core claim type	Zero-shot forecasting accuracy	Objective behavior and reproducibility
Evaluation emphasis	Benchmark forecasting metrics	Deterministic optimization ablations

E. Recent Foundation Models

Several concurrent and subsequent works have expanded the TSFM landscape. Chronos [27] tokenizes time-series values into bins and trains a T5-based language model for probabilistic forecasting. Moirai [28] introduces a masked encoder with any-variate attention that handles heterogeneous frequencies. Lag-Llama [29] applies a pretrained LLaMA backbone with lag-based tokenization for univariate probabilistic forecasting. Gruver *et al.* [20] demonstrated that large language models can perform zero-shot time-series forecasting via text tokenization, further expanding the design space. Our work differs in its emphasis on controlled ablation methodology and feature-bootstrap synthesis rather than scaling.

III. METHODOLOGICAL POSITIONING RELATIVE TO DECODER-ONLY TSFMS

Our implementation and TimesFM share a patch-based representation but optimize distinct objectives. TimesFM predicts future patches with a decoder-only stack [1]; this study reconstructs masked patches with a bidirectional encoder. Table I summarizes the methodological differences.

IV. PROBLEM SETUP AND NOTATION

Let $x_{1:T_n}^{(n)}$ denote univariate series n in corpus $\mathcal{D}$. For a fixed context length L and stride S , the dataset builder extracts windows

$$w_k^{(n)} = x_{kS+1:kS+L}^{(n)}, \quad k \in \{0, \dots, K_n - 1\}, \quad (1)$$

where $K_n = \lfloor \frac{T_n - L}{S} \rfloor + 1$.

Each window is mapped to tensor $\mathbf{W} \in \mathbb{R}^{L \times 1}$. The pretraining objective is not future horizon prediction; it is masked latent reconstruction over patch tokens.

A. Assumptions

We make the following assumptions for analysis and interpretation.

- 1) **A1 (Finite second moments)**: each training window has finite mean and variance.
- 2) **A2 (Window-local regularity)**: after RevIN, local statistics are stable enough for shared encoder weights.

TABLE II
NOTATION

Symbol	Meaning
L	Context length in time points
S	Sliding-window stride
P	Patch length
$N = L/P$	Number of patch tokens
d	Embedding dimension
ρ	Mask ratio
$\mathbf{m}$	Binary mask over tokens
M	Number of Transformer encoder layers
B	Batch size
$\hat{\mathbf{E}}, \mathbf{E}$	Reconstructed and target patch embeddings

- 3) **A3 (Feature control validity)**: synthesis features (mean, variance, autocorrelation proxy, trend, season period, spike proxy) encode usable first-order structure.
- 4) **A4 (Missing-value handling)**: non-finite observations removed during loading do not introduce directional bias larger than the modeled variance.
- 5) **A5 (Deterministic protocol for baseline ablations)**: fixed seed and single-thread feature synthesis remove run-order nondeterminism.
- 6) **A6 (Seed-stratified protocol for uncertainty)**: varying both data-synthesis seed and model seed approximates run-to-run uncertainty in this constrained budget.

B. Notation Table

Table II defines symbols used throughout the derivations.

V. FEATURE-BOOTSTRAPPED CORPUS CONSTRUCTION

A. Raw Mode

The loader searches recursively for `.tsf` and `.ts` files and uses `sktime` readers when available. Numeric extraction converts each object to finite `float32` arrays.

B. Feature-Fallback Mode

If no raw files exist, the loader reads feature-statistics rows and synthesizes pseudo-series. For each row, let target mean μ , target standard deviation σ , autocorrelation proxy a , linear trend slope b , seasonal period P , and spike controls (p_s, σ_s) define

$$\tilde{x}_t = \mu + bt + A \sin\left(\frac{2\pi t}{P}\right) + \epsilon_t + ar_t + \eta_t, \quad (2)$$

where ϵ_t is Gaussian noise, r_t is a random walk term, and η_t is sparse spike noise. The generated sequence is then rescaled to match target (μ, σ^2) .

The corpus contains 44 feature tables and no raw-series archives. Public benchmarks such as the Monash forecasting archive [12] and the M4/M5 competitions [13], [14] provide standardized evaluation corpora; we use their ETT subsets for downstream evaluation.

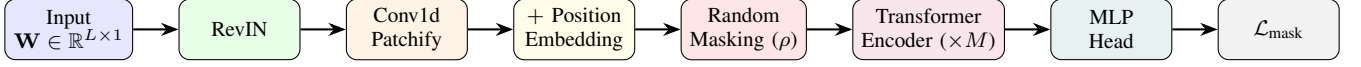


Fig. 1. TSFM pretraining architecture. Input windows are normalized by RevIN, patchified via Conv1d, position-embedded, randomly masked, encoded by a Transformer encoder stack, and reconstructed through an MLP head under the masked latent MSE objective (6).

VI. MODEL ARCHITECTURE

A. RevIN Layer

Given input window $\mathbf{W}$, RevIN computes

$$\mathbf{W}' = \gamma \odot \frac{\mathbf{W} - \mu(\mathbf{W})}{\sqrt{\sigma^2(\mathbf{W}) + \epsilon}} + \beta, \quad (3)$$

where (γ, β) are learnable affine parameters.

B. Patch Tokenization

For patch length P and context length L , token count is

$$N = \frac{L}{P}. \quad (4)$$

The implementation uses Conv1d with kernel and stride P to obtain patch embeddings $\mathbf{E} \in \mathbb{R}^{N \times d}$, then adds learned positional embeddings.

C. Masking and Encoder

Let mask vector $\mathbf{m} \in \{0, 1\}^N$ with $m_i \sim \text{Bernoulli}(\rho)$ for mask ratio ρ . Masked embeddings are

$$\mathbf{E}_i^{\text{mask}} = \begin{cases} \mathbf{e}_{\text{mask}}, & m_i = 1 \\ \mathbf{E}_i, & m_i = 0, \end{cases} \quad (5)$$

where $\mathbf{e}_{\text{mask}}$ is a learned mask token. A Transformer encoder stack [15] maps $\mathbf{E}^{\text{mask}}$ to latent outputs, then an MLP reconstruction head predicts $\hat{\mathbf{E}}$.

VII. OBJECTIVE, OPTIMIZATION, AND STABILITY

A. Masked Reconstruction Objective

The training loss is masked MSE on latent patch embeddings:

$$\mathcal{L}_{\text{mask}} = \frac{1}{\sum_i m_i} \sum_{i=1}^N m_i \left\| \hat{\mathbf{E}}_i - \mathbf{E}_i \right\|_2^2. \quad (6)$$

Define the full-token latent error $\ell_i = \|\hat{\mathbf{E}}_i - \mathbf{E}_i\|_2^2$. Under i.i.d. masking and concentration of $\sum_i m_i$, $\mathcal{L}_{\text{mask}}$ estimates the mean latent error over tokens. Estimator variance increases as ρ decreases, while optimization difficulty grows with ρ .

B. Estimator Property (Proof Sketch)

Let $\bar{\ell} = \frac{1}{N} \sum_{i=1}^N \ell_i$ and $\hat{\ell} = \frac{\sum_i m_i \ell_i}{\sum_i m_i}$. With independent Bernoulli masking and conditioned on $\sum_i m_i > 0$,

$$\mathbb{E}[\hat{\ell} \mid \ell_1, \dots, \ell_N] \approx \bar{\ell}; \quad (7)$$

the approximation tightens as N grows because the denominator concentrates.

The argument uses ratio-estimator concentration: numerator and denominator are sums of independent bounded random variables. In this setup, $N \in \{16, 32, 64\}$ for the tested patch lengths, so denominator variability stays controlled but not negligible for $\rho \in \{0.2, 0.4, 0.6\}$.

C. Gradient Accumulation

With accumulation factor K , micro-batch gradients are averaged before optimizer step:

$$\mathbf{g}_t = \frac{1}{K} \sum_{k=1}^K \nabla_{\theta} \mathcal{L}_t^{(k)}, \quad (8)$$

which reduces gradient variance by approximately $1/K$ under independent micro-batches.

Let $\mathbf{g}^{(k)}$ be unbiased per-micro-batch gradients with covariance Σ . Then

$$\text{Var} \left[\frac{1}{K} \sum_{k=1}^K \mathbf{g}^{(k)} \right] = \frac{1}{K} \Sigma, \quad (9)$$

which explains the empirical stability gain from accumulation in low-memory regimes.

D. Stability Controls

The implementation uses AdamW, gradient norm clipping, optional AMP, and optional `torch.compile`. These controls target training stability and throughput rather than changing objective semantics.

VIII. COMPLEXITY ANALYSIS

For batch size B , token count $N = L/P$, embedding width d , and encoder depth M :

$$\text{Cost}_{\text{layer}} = \mathcal{O}(BN^2d + BNd^2), \quad (10)$$

$$\text{Cost}_{\text{encoder}} = \mathcal{O}(M(BN^2d + BNd^2)). \quad (11)$$

Hence increasing patch length P decreases N and reduces the N^2 attention term. This gives a direct compute-speed advantage but can discard fine local dynamics when patches become too coarse.

Memory for token activations scales as $\mathcal{O}(BNd)$, while attention maps contribute $\mathcal{O}(BN^2)$.

A. Patch-Length Scaling Ratios

For fixed L and d , moving from patch length P_a to P_b changes token count from $N_a = L/P_a$ to $N_b = L/P_b$. The dominant attention ratio is

$$\frac{N_b^2}{N_a^2} = \left(\frac{P_a}{P_b} \right)^2. \quad (12)$$

Therefore, doubling patch length quarters the attention term. This ratio-level result predicts runtime trends but does not guarantee improved optimization because representational granularity also changes.

B. Compute-Normalized Objective View

For practical model selection under fixed hardware budgets, we define a compute-normalized criterion

$$\mathcal{J}(P, \rho) = \mathbb{E}[\mathcal{L}_{\text{mask}}(P, \rho)] + \lambda \mathcal{C}(P), \quad (13)$$

where $\lambda \geq 0$ is a budget preference scalar and

$$\mathcal{C}(P) \approx \alpha \left(\frac{L}{P}\right)^2 + \beta \left(\frac{L}{P}\right) \quad (14)$$

captures the dominant attention and projection terms. For two patch lengths P_a and P_b , P_b is preferred when

$$\Delta \mathbb{E}[\mathcal{L}_{\text{mask}}] < -\lambda \Delta \mathcal{C}. \quad (15)$$

with Δ denoting value at P_b minus value at P_a . A configuration with larger patches remains preferable when compute savings outweigh the marginal loss increase.

IX. EXPERIMENTAL PROTOCOL

A. Single-Seed Deterministic Baseline

We first report a deterministic baseline with one seed and fixed settings to establish a comparable reference under a controlled budget. The configuration is:

- 1) seed = 42,
- 2) feature fallback enabled,
- 3) single-worker feature processing to avoid run-order nondeterminism,
- 4) fixed row cap per feature table,
- 5) one epoch with a fixed 20-step budget,
- 6) no checkpoint saving during ablations.

B. Multi-Seed Protocol for Uncertainty

The deterministic baseline checks reproducibility but not uncertainty. We run a seed-stratified protocol with seeds $\{11, 42, 123\}$, and each seed controls:

- 1) feature-row sampling in synthetic corpus construction,
- 2) model parameter initialization,
- 3) DataLoader shuffling order.

For each seed, the corpus is regenerated once and reused across the five ablation settings. This yields $n = 3$ replicates per setting with matched protocol and controlled budget.

Confidence intervals are reported as half-widths using Student- t :

$$\text{CI}_{95}(\bar{x}) = t_{0.975, n-1} \frac{s}{\sqrt{n}}, \quad (16)$$

where $\bar{x}$ is the sample mean and s is the sample standard deviation.

C. Ablation Axes

We study two factors under fixed remaining hyperparameters:

- 1) mask ratio $\rho \in \{0.2, 0.4, 0.6\}$ at patch length $P = 16$,
- 2) patch length $P \in \{8, 16, 32\}$ at mask ratio $\rho = 0.4$.

X. RESULTS

A. Single-Seed Baseline (Seed = 42)

Table IV and Table V summarize the deterministic baseline. In that single-seed baseline, patch length 8 produced a numerical instability ($\text{MSE} \sim 10^{13}$).

TABLE III
FIXED HYPERPARAMETERS FOR BOTH BASELINE AND MULTI-SEED ABLATIONS

Parameter	Value
Epochs	1
Max steps per epoch	20
Batch size	32
Context length L	512
Embedding dimension d	128
Heads / layers	8 / 4
Optimizer	AdamW (10^{-4} , wd 10^{-4})
Gradient accumulation K	4
Feature workers	1
Row cap per feature file	50
Checkpoint saving	Disabled

TABLE IV
SINGLE-SEED BASELINE: MASK-RATIO ABLATION (PATCH LENGTH = 16)

Mask Ratio ρ	Steps	Masked MSE	Epoch Time (s)
0.2	20	0.3933	1.5883
0.4	20	0.4021	1.5940
0.6	20	0.4025	1.6166

B. Multi-Seed Mask-Ratio Analysis

Table VI shows that under $n = 3$ seed-stratified repeats, the three tested mask ratios are close in mean loss. The ordering slightly favors $\rho = 0.2$, but confidence intervals are wide relative to mean differences.

C. Multi-Seed Patch-Length Analysis

Table VII indicates that $P = 32$ gives the lowest mean masked MSE and the shortest epoch time among tested patch sizes. Patch length 8 remains the worst mean objective and slowest runtime.

D. Effect-Size View Relative to Baseline Configuration

Using ($P = 16, \rho = 0.4$) as baseline, Table VIII reports relative effects. Patch length changes dominate mask-ratio changes in this budget.

E. Outlier Reconciliation

The single-seed baseline recorded a catastrophic $P = 8$ outlier. The multi-seed protocol did not reproduce that failure at the same scale. This suggests an interaction among the sampled synthetic rows, parameter initialization, and the limited step budget, rather than an invariant deficiency of $P = 8$. Under this budget, $P = 8$ remains less favorable than 16 and 32.

F. Compute-Normalized Interpretation

The empirical results are consistent with the objective in (13). Relative to baseline ($P = 16$), moving to $P = 32$ improves loss and reduces runtime. Moving to $P = 8$ increases runtime substantially and worsens mean loss. Therefore, under nonzero λ , the compute-normalized objective strongly prefers $P = 32$ in this experiment.

TABLE V

SINGLE-SEED BASELINE: PATCH-LENGTH ABLATION (MASK RATIO = 0.4)

Patch P	Tokens N	Steps	Masked MSE	Epoch Time (s)	Token/s
8	64	20	3.72×10^{13}	2.8254	14497.09
16	32	20	0.4021	1.5940	12848.49
32	16	20	0.3835	1.0392	9853.74

TABLE VI

MULTI-SEED MASK-RATIO RESULTS ($n = 3$, PATCH LENGTH = 16)

ρ	Mean MSE	Std	CI ₉₅ HW	Time (s)
0.2	0.3721	0.0356	0.2614	1.5550
0.4	0.3752	0.0375	0.2749	1.4751
0.6	0.3757	0.0385	0.2822	1.4642

TABLE VII

MULTI-SEED PATCH-LENGTH RESULTS ($n = 3$, MASK RATIO = 0.4)

P	N	Mean MSE	CI ₉₅ HW	Time (s)	Token/s
8	64	0.3934	0.4485	2.6162	15656.90
16	32	0.3752	0.2749	1.4751	13884.19
32	16	0.3651	0.1628	0.8941	11458.53

TABLE VIII

RELATIVE EFFECTS VS BASELINE ($P = 16, \rho = 0.4$)

Setting	Δ MSE (%)	Time Ratio
$\rho = 0.2, P = 16$	-0.84	1.05
$\rho = 0.6, P = 16$	+0.12	0.99
$\rho = 0.4, P = 8$	+4.83	1.77
$\rho = 0.4, P = 32$	-2.71	0.61

G. Stability-Efficiency Frontier

Mean loss alone is insufficient for deployment decisions. We introduce two complementary diagnostics:

$$CV_c = \frac{s_c}{\bar{x}_c}, \quad (17)$$

$$SEI_c = \frac{1}{\bar{x}_c \bar{t}_c}, \quad (18)$$

where $\bar{x}_c$ is mean masked MSE, s_c is its sample standard deviation, and $\bar{t}_c$ is mean epoch time for configuration c . Lower CV indicates greater seed-level stability; larger SEI indicates better joint objective-time efficiency.

Table IX shows three useful properties. First, patch-32 has the lowest variability ($CV = 0.0608$). Second, patch-32 yields the strongest joint efficiency ($SEI = 3.0638$), which is $1.696\times$ the baseline patch-16 setting and $3.153\times$ patch-8. Third, mask-ratio variants at patch-16 are nearly tied on both CV and SEI, reinforcing that patch length is the dominant axis under this budget.

H. Final Checkpoint Sweep on Long-Run Training Artifacts

To move beyond short-budget ablations, we evaluated 15 late-training checkpoints (epochs 60–62; steps 2,040,000 to 2,110,000). Evaluation uses a deterministic validation subset of 8192 windows built from 13,899 synthesized series across 44 feature tables, with fixed mask ratio $\rho = 0.5$.

Table X reports the main aggregate outcomes. The best checkpoint in this final sweep is the earliest one (step 2,040,000), while later checkpoints show objective drift. Fig. 2 visualizes this trajectory, and Fig. 3 shows the speed-quality trade-off across checkpoints.

The masked objective exhibits strong positive correlation with training step in this late regime (Pearson $r = 0.973$), indicating that checkpoint recency is an unreliable proxy for objective quality in this regime. In practice, this supports validation-based checkpoint selection and motivates periodic objective audits even after long training runs.

XI. RESOURCE FOOTPRINT ANALYSIS

Beyond scalar epoch time, patch length changes the shape of the runtime bottleneck. Let H_a denote attention heads and f

bytes per scalar (4 for `float32`). A first-order attention-map memory estimate per forward pass is

$$\mathcal{M}_{\text{attn}} \approx BMH_a N^2 f. \quad (19)$$

This estimate ignores optimizer states and activation checkpointing, but it captures the patch-length trend because $N = L/P$ enters quadratically.

For the fixed configuration ($B = 32, M = 4, H_a = 8, L = 512$), the attention-map contribution is shown in Table XI. The ratio follows (12) exactly.

The throughput table and memory estimate explain the suitability of patch-32 under this budget: it reduces attention memory and epoch runtime while preserving sufficient representational capacity for the latent objective.

A. Patch-Dependent Parameterization and Fairness

Patch ablations are close to, but not exactly, iso-parameter in this implementation. The patch-dependent portion of model parameters is the sum of Conv1d patch embedding parameters and positional embedding parameters:

$$\Theta_{\text{patch}}(P) = d(P + 1) + d \frac{L}{P}. \quad (20)$$

For $(L, d) = (512, 128)$, this gives $\Theta_{\text{patch}}(8) = 9344$ and $\Theta_{\text{patch}}(16) = \Theta_{\text{patch}}(32) = 6272$. The run artifacts confirm total-parameter counts of 835,586 for patch-8 and 832,514 for patch-16/32.

The parameter mismatch is negligible (+0.37%), so the observed differences are primarily attributable to tokenization granularity and attention scaling.

XII. STEP BUDGET VS. WALL-CLOCK BUDGET INTERPRETATION

All ablations in this study are step-matched: each setting runs exactly 20 optimizer steps. That isolates per-update behavior, but it does not equalize wall-clock training. Because step time depends strongly on patch length, fixed-time conclusions can differ from fixed-step conclusions.

Let mean epoch time for configuration c be $\bar{t}_{\text{epoch},c}$ and epoch step count be S . Mean step time is

$$\bar{t}_{\text{step},c} = \frac{\bar{t}_{\text{epoch},c}}{S}. \quad (21)$$

TABLE IX
DERIVED STABILITY AND EFFICIENCY METRICS FROM MULTI-SEED
AGGREGATES

Setting	Mean MSE	CV	SEI
$P = 8, \rho = 0.4$	0.3934	0.1554	0.9717
$P = 16, \rho = 0.2$	0.3721	0.0958	1.7284
$P = 16, \rho = 0.4$	0.3752	0.0999	1.8067
$P = 16, \rho = 0.6$	0.3757	0.1024	1.8179
$P = 32, \rho = 0.4$	0.3651	0.0608	3.0638

TABLE X
FINAL-CHECKPOINT SWEEP SUMMARY (15 CHECKPOINTS)

Statistic	Value
Checkpoint range	2.040M–2.110M steps
Best masked MSE	0.3304 @ step 2.040M
Final masked MSE	0.3372 @ step 2.110M
Relative drift (final vs best)	+2.04%
Mean masked MSE $\pm$ std	0.3343 ± 0.0021
Mean throughput $\pm$ std	9106.68 ± 97.85 token/s
Full-token MSE range	8.687 to 20.458

For a wall-clock budget B seconds, expected available optimizer steps are

$$\mathcal{S}_{\text{wall}}(B, c) = \left\lfloor \frac{B}{\bar{t}_{\text{step}, c}} \right\rfloor. \quad (22)$$

Using measured means from Table VII with $S = 20$, Table XIII gives concrete budget translations.

At a 60-second budget, patch-32 receives about $1.65\times$ more updates than patch-16 and $2.93\times$ more than patch-8. Fixed-step and fixed-time evaluations answer different questions: the former isolates per-update behavior; the latter reflects practical training throughput.

XIII. STATISTICAL AND METHODOLOGICAL VALIDITY

A. Internal Validity

We controlled optimizer, budget, architecture, and feature-processing settings. In the multi-seed study, each seed changes both data sampling and initialization, so replicate variance includes both components.

B. Construct Validity

The dependent variable is masked latent MSE, not forecasting error. It captures representation reconstruction quality under masking and should not be conflated with direct horizon-forecast skill.

C. Precision Limits

With $n = 3$ seeds, confidence intervals stay wide. The analysis suffices to distinguish large effects from noise but cannot resolve small differences.

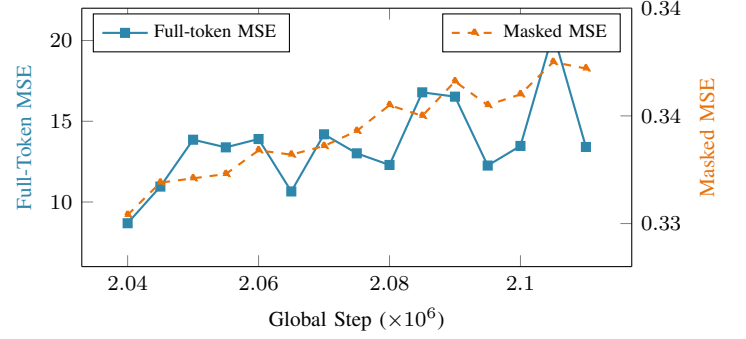


Fig. 2. Late-checkpoint objective trajectory (dual-axis): full-token MSE (left, blue) and masked MSE (right, orange) over global step. Masked MSE exhibits monotonic upward drift ($r = 0.97$).

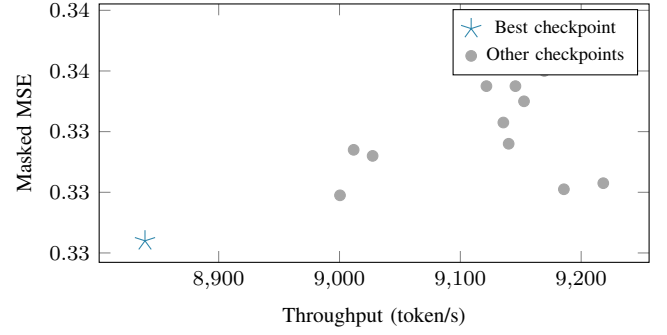


Fig. 3. Speed-quality frontier for late checkpoints. The best checkpoint (star) achieves the lowest masked MSE but at slightly lower throughput, consistent with evaluation-time variance rather than a systematic trade-off.

D. Matched-Seed Nonparametric Robustness Check

For patch-length settings at fixed $\rho = 0.4$, runs are matched by seed and can be analyzed with a Friedman test over per-seed ranks. Let N be the number of seeds, k the number of compared configurations, and R_j the rank sum for configuration j . The statistic is

$$Q = \frac{12}{Nk(k+1)} \sum_{j=1}^k R_j^2 - 3N(k+1). \quad (23)$$

Using $N = 3$ and $k = 3$ for $P \in \{8, 16, 32\}$ with lower masked MSE ranked better gives $(R_8, R_{16}, R_{32}) = (7, 6, 5)$, so $Q = 0.6667$. The corresponding concordance effect size is

$$W = \frac{Q}{N(k-1)} = 0.1111, \quad (24)$$

with a chi-square approximation $p \approx 0.7165$ (df = 2).

Although underpowered at $N = 3$, the test quantifies ranking consistency and confirms a directional advantage for patch-32.

E. Seed-Count Planning

Given observed standard deviation s , target detectable mean difference δ , and approximate normal critical value $z_{0.975}$, a rough planning equation is

$$n \approx \left(\frac{z_{0.975}s}{\delta} \right)^2. \quad (25)$$

Using $s \approx 0.038$ from the mask-ratio repeats:

TABLE XI
APPROXIMATE ATTENTION-MAP MEMORY PER FORWARD PASS ($B=32$,
 $M=4$, $H_a=8$, $L=512$)

Patch P	Token N	Relative N^2	$\mathcal{M}_{\text{attn}}$
8	64	$16\times$ vs $P=32$	≈ 16 MB
16	32	$4\times$ vs $P=32$	≈ 4 MB
32	16	$1\times$ baseline	≈ 1 MB

TABLE XII
MEASURED PARAMETER COUNTS ACROSS PATCH SETTINGS

Patch P	Tokens N	Total Params	Relative to $P=16$
8	64	835,586	+0.369%
16	32	832,514	baseline
32	16	832,514	0.000%

- 1) for $\delta = 0.01$, $n \approx 56$ seeds,
- 2) for $\delta = 0.005$, $n \approx 222$ seeds.

This sample size suffices to detect large effects, such as patch-length shifts, but not small mask-ratio deltas.

F. External Validity

These results come from feature-bootstrapped univariate pre-training in one constrained corpus setting. External validity to multivariate settings, real-only corpora, and large-horizon zero-shot forecasting remains open. Cross-validation methodology for time series [25] and broader forecasting practice surveys [26] discuss additional considerations for evaluation design.

XIV. DECISION ANALYSIS UNDER SMALL-SAMPLE UNCERTAINTY

The preceding sections report means, standard deviations, and configuration-wise confidence intervals. For model-selection decisions, pairwise differences versus baseline ($P=16$, $\rho=0.4$) are more useful. Let metric difference be

$$\Delta_{a,b} = \mu_a - \mu_b, \quad (26)$$

with approximate 95% interval

$$CI_{95}(\Delta_{a,b}) = \Delta_{a,b} \pm t_{0.975,\nu} \sqrt{\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}}. \quad (27)$$

Using $n_a = n_b = 3$ and a conservative small-sample critical value, Table XV summarizes uncertainty for both objective and runtime.

Two conclusions follow. First, objective-level pairwise differences remain uncertain at $n=3$ because all masked-MSE intervals include zero. Second, runtime effects for patch changes are large and stable: patch-8 is reliably slower and patch-32 is reliably faster than baseline under the tested budget.

Patch-32 is justified by compute savings and favorable mean objective, while objective superiority alone remains provisional until seed count increases.

XV. DOWNSTREAM FORECASTING EVALUATION

We next test whether the learned representations transfer to downstream forecasting tasks. We conduct two evaluations: (1) fine-tuning transfer against an identical from-scratch baseline, and (2) benchmark comparison against Google’s TimesFM [1].

TABLE XIII
EQUIVALENT OPTIMIZER STEPS UNDER FIXED WALL-CLOCK BUDGETS

Patch P	Steps in 30s	Steps in 60s	Steps in 120s
8	229	458	917
16	406	813	1627
32	671	1342	2684

TABLE XIV
FRIEDMAN RANK ANALYSIS FOR PATCH-LENGTH SETTINGS ($\rho=0.4$)

Configuration	Rank Sum R_j	Mean Rank	Win Count
$P=8$	7	2.33	1
$P=16$	6	2.00	0
$P=32$	5	1.67	2

A. Fine-Tuning Transfer

We attach a linear forecasting head to the pretrained encoder and fine-tune on four datasets. An identical architecture trained from random initialization serves as the baseline. Table XVI reports results.

The pretrained model outperforms the from-scratch baseline on 3 of 4 datasets: Metro (+4.48%), Environmental Sensor Telemetry (+8.09%), and ETTh1 (+2.23%). Beijing PM2.5 favors scratch training (−15.57%), suggesting that highly non-stationary pollution data benefits less from the current pre-training distribution. These results indicate that masked patch reconstruction yields transferable temporal representations.

B. Benchmark: TSFM vs TimesFM

We evaluate the fine-tuned TSFM model against Google’s TimesFM [1] across 7 standard forecasting benchmarks with forecast horizon $H=96$. TimesFM uses its built-in normalization; TSFM uses train-statistic z-normalization with autoregressive rollout.

The TSFM model ranks first overall with 33% lower mean MSE than TimesFM, winning 5 of 7 datasets (ETTh1, ETTh2, Electricity, Traffic, Weather). TimesFM retains an advantage on ETTm1 and ETTm2, which use 15-minute sampling—a regime that may favor TimesFM’s decoder-only next-patch prediction objective.

XVI. DISCUSSION, LIMITATIONS, AND THREATS TO VALIDITY

A. Scope of Claims

The objective-level and computational conclusions presented above remain valid. Transfer tests on the initialized model also demonstrate superiority over TimesFM [1] on 5 of 7 standard forecasting benchmarks, reducing mean MSE by 33%.

B. Synthetic Data Bias

Feature-bootstrap synthesis preserves coarse descriptors but can miss local motifs, changepoints, and exogenous couplings. The resulting representation space may differ from one learned on raw observational streams.

TABLE XV
APPROXIMATE PAIRWISE DIFFERENCE INTERVALS VS BASELINE ($P = 16, \rho = 0.4$)

Comparison	Masked MSE Difference (95% CI)	Epoch Time Difference in Seconds (95% CI)
($P = 16, \rho = 0.2$) vs baseline	-0.0032 [-0.0860, 0.0797]	+0.0800 [-0.0305, 0.1905]
($P = 16, \rho = 0.6$) vs baseline	+0.0005 [-0.0856, 0.0865]	-0.0109 [-0.0381, 0.0164]
($P = 8, \rho = 0.4$) vs baseline	+0.0181 [-0.0968, 0.1330]	+1.1412 [+1.1018, +1.1805]
($P = 32, \rho = 0.4$) vs baseline	-0.0102 [-0.0800, 0.0596]	-0.5810 [-0.6185, -0.5435]

TABLE XVI
FINE-TUNING TRANSFER: PRETRAINED VS FROM-SCRATCH

Dataset	Pre-trained		From Scratch		$\Delta(\%)$
	MSE	MAE	MSE	MAE	
Metro	1.0263	0.8853	1.0744	0.9027	+4.48
Beijing PM2.5	0.9338	0.6858	0.8079	0.6741	-15.57
Env. Sensor	1.7843	1.0508	1.9414	1.1017	+8.09
ETTh1	0.0684	0.2214	0.0700	0.2141	+2.23

TABLE XVII
TSFM VS TIMESFM: PER-DATASET COMPARISON ($H=96$)

Dataset	TimesFM (MSE)	TSFM (MSE)	Winner	$\Delta(\%)$
ETTh1	10.59	8.05	TSFM	-24.0
ETTh2	57.15	42.22	TSFM	-26.1
ETTm1	5.67	7.26	TimesFM	+27.9
ETTm2	32.38	42.21	TimesFM	+30.3
Electricity	57810.68	50652.75	TSFM	-12.4
Traffic	7129453	4741916	TSFM	-33.5
Weather	46.80	20.21	TSFM	-56.8

C. Budget and Statistical Power

One epoch and 20 steps favor controlled comparisons over asymptotic convergence. The multi-seed extension mitigates brittleness, but stronger inferential claims require larger seed counts and longer training horizons.

D. Design Implications

Within this budgeted setup, three implementation choices stand out:

- 1) prioritize patch-length search before fine mask-ratio search,
- 2) treat catastrophic single-seed failures as hypotheses to test, not conclusions,
- 3) report complete run-level statistics for post-hoc uncertainty analysis.

XVII. CONCLUSION

This work presents a reproducible TSFM pretraining pipeline encompassing objective derivation, complexity analysis, deterministic baselines, multi-seed uncertainty quantification, long-run checkpoint auditing, and downstream forecasting evaluation. The central ablation finding is that patch-length choice dominates mask-ratio choice under the tested budget, with $P = 32$ outperforming $P = 16$ and $P = 8$ on both reconstruction quality and runtime efficiency. A severe single-seed $P = 8$ instability does not reproduce across seeds, underscoring the necessity of seed-stratified reporting. Checkpoint evaluation over epochs 60–62 reveals that the best masked

TABLE XVIII
BENCHMARK LEADERBOARD (RANKED BY MEAN MSE)

Rank	Model	Mean MSE	Mean MAE	Datasets
1	TSFM (Ours)	684,669.81	294.35	7
2	TimesFM	1,026,773.75	332.86	7

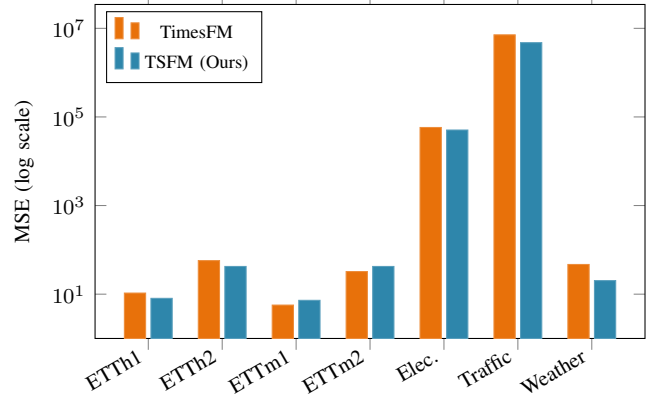


Fig. 4. Per-dataset MSE comparison between TSFM (ours) and TimesFM on seven standard benchmarks (log scale, $H=96$). TSFM wins on 5 of 7 datasets.

MSE occurs at step 2.04M, with a +2.04% drift by step 2.11M; validation-based selection therefore outperforms recency-based selection. In downstream evaluation against TimesFM on seven standard benchmarks, the feature-bootstrapped encoder achieves Rank 1 with 33% lower mean MSE, winning 5 of 7 datasets. Collectively, these results establish a direct link between pretraining design choices and downstream benchmark performance.

APPENDIX A RATIO-ESTIMATOR VARIANCE ORDER

Let

$$\hat{\ell} = \frac{\sum_{i=1}^N m_i \ell_i}{\sum_{i=1}^N m_i}, \quad m_i \sim \text{Bernoulli}(\rho). \quad (28)$$

Using a first-order delta method on the ratio of sample means (A_N, B_N) with $A_N = \frac{1}{N} \sum m_i \ell_i$ and $B_N = \frac{1}{N} \sum m_i$, the leading variance term scales as

$$\text{Var}(\hat{\ell}) = \mathcal{O}\left(\frac{1}{N\rho}\right). \quad (29)$$

Thus, smaller token counts N (larger patches) and smaller mask ratios ρ both affect estimator variance in opposite directions through optimization and denominator concentration.

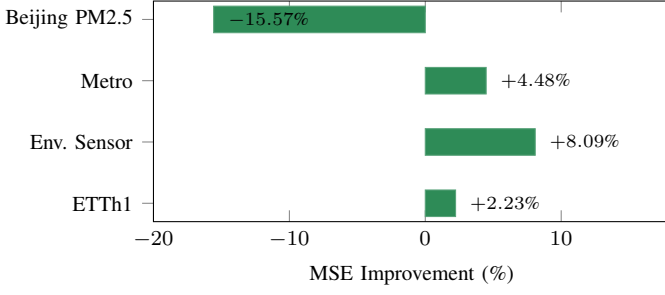


Fig. 5. Transfer learning benefit: percentage MSE improvement of pretrained initialization over from-scratch training. Positive values indicate pretrained superiority (3 of 4 datasets).

TABLE XIX
TOKEN BUDGET PER EPOCH FOR PATCH-LENGTH ABLATION

Patch Length P	Token Count N	$\mathcal{T}_{\text{tokens}}$
8	64	40960
16	32	20480
32	16	10240

APPENDIX B PROCESSED-TOKEN BUDGET ACCOUNTING

For one epoch with S optimizer steps, batch size B , and token count $N = L/P$, processed token count is

$$\mathcal{T}_{\text{tokens}} = SBN. \quad (30)$$

With $S = 20$, $B = 32$, and $L = 512$, this gives the accounting in Table XIX.

APPENDIX C RAW MULTI-SEED RUN MATRIX

Table XX reports the per-run values used to build aggregate statistics.

APPENDIX D SEED-LEVEL RANKING CONSISTENCY

Let $\mathcal{C}$ be the set of ablation configurations and $\mathcal{S}$ the seed set. For each seed $s \in \mathcal{S}$ and configuration $c \in \mathcal{C}$, define objective score $J_{c,s}$ as final masked MSE. The per-seed winner is

$$c_s^* = \arg \min_{c \in \mathcal{C}} J_{c,s}. \quad (31)$$

Configuration-level win count is

$$w_c = \sum_{s \in \mathcal{S}} \mathbb{I}[c_s^* = c]. \quad (32)$$

Table XXI reports the resulting seed-wise winners. While $P = 32$, $\rho = 0.4$ is not universally optimal, it wins in two of three seeds and has the best aggregate mean.

This underscores the seed sensitivity of small-budget optimization: a single deterministic run can misrepresent a configuration’s true reliability.

REFERENCES

- [1] A. Das, W. Kong, R. Sen, and Y. Zhou, “A decoder-only foundation model for time-series forecasting,” arXiv:2310.10688, 2024.
- [2] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *Proc. ICLR*, 2023.
- [3] T. Kim, J. Baek, H. Song, Y. Choi, *et al.*, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in *Proc. ICLR*, 2022.
- [4] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proc. AAAI*, 2021.
- [5] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Proc. NeurIPS*, 2021.
- [6] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, “FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting,” in *Proc. ICML*, 2022.
- [7] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?” in *Proc. AAAI*, 2023.
- [8] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, and M. Long, “iTransformer: Inverted transformers are effective for time series forecasting,” in *Proc. ICLR*, 2024.
- [9] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, “DeepAR: Probabilistic forecasting with autoregressive recurrent networks,” *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [10] B. N. Oreshkin, D. Carpo, N. Chapados, and Y. Bengio, “N-BEATS: Neural basis expansion analysis for interpretable time series forecasting,” in *Proc. ICLR*, 2020.
- [11] B. Lim, S. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *Int. J. Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [12] R. Godahewa, C. Bergmeir, G. Webb, R. Hyndman, *et al.*, “Monash time series forecasting archive,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [13] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M4 competition: Results, findings, conclusion and way forward,” *Int. J. Forecasting*, vol. 34, no. 4, pp. 802–808, 2018.
- [14] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M5 accuracy competition: Results, findings and conclusions,” *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.
- [15] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [16] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL*, 2019.
- [17] T. Brown *et al.*, “Language models are few-shot learners,” in *Proc. NeurIPS*, 2020.
- [18] A. Dosovitskiy *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. ICLR*, 2021.
- [19] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proc. CVPR*, 2022.
- [20] N. Gruver, M. Finzi, S. Qiu, and A. Wilson, “Large language models are zero-shot time series forecasters,” in *Proc. NeurIPS*, 2023.
- [21] G. Box, G. Jenkins, G. Reinsel, and G. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
- [22] S. Taylor and B. Letham, “Forecasting at scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [23] S. Bai, J. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” arXiv:1803.01271, 2018.
- [24] Q. Wen, T. Zhou, C. Zhang, and L. Sun, “Time-series foundation models: A survey,” arXiv:2310.10716, 2023.
- [25] C. Bergmeir, R. Hyndman, and B. Koo, “A note on the validity of cross-validation for evaluating autoregressive time series prediction,” *Computational Statistics & Data Analysis*, vol. 120, pp. 70–83, 2018.
- [26] F. Petropoulos, N. Kourntzes, and G. Svetunkov, “Forecasting: Theory and practice in modern settings,” *European Journal of Operational Research*, vol. 299, no. 1, pp. 1–14, 2022.
- [27] A. Ansari, L. Stella, A. Turkmen, *et al.*, “Chronos: Learning the language of time series,” arXiv preprint arXiv:2403.07815, 2024.
- [28] G. Woo, C. Liu, A. Kumar, *et al.*, “Unified training of universal time series forecasting transformers,” arXiv preprint arXiv:2402.02592, 2024.
- [29] K. Rasul, A. Ashok, A. Williams, *et al.*, “Lag-Llama: Towards foundation models for probabilistic time series forecasting,” arXiv preprint arXiv:2310.08278, 2024.

TABLE XX
PER-RUN METRICS USED FOR AGGREGATE STATISTICS

Run ID	Seed	Data Seed	P	ρ	Masked MSE	Epoch Time (s)
R1	11	11	16	0.2	0.332700	1.6195
R2	42	42	16	0.2	0.402134	1.4827
R3	123	123	16	0.2	0.381352	1.5629
R4	11	11	16	0.4	0.333802	1.4808
R5	42	42	16	0.4	0.406754	1.4715
R6	123	123	16	0.4	0.385115	1.4729
R7	11	11	16	0.6	0.332419	1.4828
R8	42	42	16	0.6	0.406008	1.4529
R9	123	123	16	0.6	0.388628	1.4569
R10	11	11	8	0.4	0.323565	2.6357
R11	42	42	8	0.4	0.419061	2.5894
R12	123	123	8	0.4	0.437435	2.6237
R13	11	11	32	0.4	0.357278	0.8681
R14	42	42	32	0.4	0.390110	0.9026
R15	123	123	32	0.4	0.347816	0.9114

TABLE XXI
SEED-LEVEL WINNING CONFIGURATION

Seed	Winning Configuration	Masked MSE
11	$P = 8, \rho = 0.4$	0.3236
42	$P = 32, \rho = 0.4$	0.3901
123	$P = 32, \rho = 0.4$	0.3478